

OPTI 536 – Introduction to Image Science

Spring, 2025

Homework 6

Assigned: April 23, 2025

Due: May 7, 2025 at 11:59 pm

In this homework, you will explore both active and passive triangulation 3D imaging principles: fringe triangulation and passive stereo. In Part 1, you will implement and analyze fringe projection triangulation algorithms including camera calibration, phase retrieval via 4 phase shift and 3D reconstruction. In Part 2, you will build your own passive stereo 3D scanner with your phone camera. Together, these parts will deepen your understanding of the difference between active and passive triangulation methods.

Feel free to discuss the homework with others in the class or the instructors. You may use any books or websites you find, but please give a brief citation for any references you use. You must write up your final solutions independently without direct copying from these sources or from solutions by other students. **Please be aware that we implemented advanced measures to check for duplicated code (also for code downloaded from the internet)!**

Please submit your solutions to the D2L website by 11:59pm on the due date. **Your submission should consist of a pdf that includes all requested figures and the answers to the questions PLUS requested code files and images (see last sub-problem for each part).**

Please do not convert your pdf to grayscale this time (color is important) and do not lock/encrypt any of your submitted files.

Start with this homework EARLY, so that we have enough time to help you with any problems. The following descriptions are intentionally lengthy to provide you with enough detail and tricks to gently guide you through the process. **Keep in mind that the length of the descriptions is not related to the actual workload you need to complete.** Start with this homework EARLY, so that we have enough time to help you with any problems.

Part 1 (50 points)

In this part, you will implement fringe projection triangulation using the provided dataset. This is also a type of stereo vision. Instead of having two identical cameras to form the stereo pair, there is one camera and one projector in this system where the projector can be treated as the inverse camera model. First, with a group of checkerboard images, you can obtain the camera intrinsic parameters. Then you need to implement the four-phase-shift algorithm to find the correspondence between camera and projector (you might reuse or modify your implementation from the Deflectometry homework). There are two purposes of establishing correspondence. One is to help calibrate the projector which is treated as inverse camera model. The other one is for point cloud reconstruction. Last, you will derive the triangulation equation to calculate the point cloud.

Question 1. Camera calibration (15 points). We capture images of checkerboard patterns with known checker sizes to relate the camera coordinate system to the “world” coordinate system. The camera reprojection matrix is $P = K [R | T]$, and it relates the world system $\hat{x}$ to the camera system x_c by $x_c = P \hat{x}$.

- a) What is the “world” coordinate system? What is its relation to the camera coordinate system (Think in terms of parameters of the camera and the optical system)? What part of $P = K [R | T]$ is intrinsic and what is extrinsic? If we image 30 checkerboards, how many P’s, K’s, R’s and T’s would we have?

Use the provided dataset “Data/Calibration/NoPattern, this consists of 30 checkerboard images for camera calibration. Use a camera calibration library in either Camera Calibration Toolbox for MATLAB or OpenCV package for Python to detect the corners of the checkerboards and perform camera calibration. The program returns the values needed to compute the camera intrinsic matrix, extrinsics and distortion parameters of our camera. **Please remove the images where not all checkerboard corners are detected, otherwise the next step would fail.** The distortion parameters define a model used to describe the nonlinear effects of the camera lens in determining imaging pixel locations on the sensor at various locations around the optical axis. Once we have these distortion parameters, we can undistort captured camera images to ensure the reprojection matrix P is a linear transformation. The checker size in the provided images is 15 mm per square.

Report the following:

- b) Plot the detected corners of the first checkerboard image in terms of camera coordinates (pixel column along ‘x’ axis and pixel row along ‘y’ axis). Were you able to successfully detect all corners in all the 30 images? If not, report what you think went wrong in detecting the checker corners. Plot the respective images where something went wrong (if applicable).
- c) What was your calibration error?
- d) Display the checkerboard planes with respect to the camera. This can be done in MATLAB using `showExtrinsics()` or if using Python, you can use the function “*showExtrinsics*,” that was provided by us.

Question 2. Projector calibration (15 points). Now we want to calibrate the projector. We do this by treating the projector as an inverse camera, which we can do by having the camera help the projector “see” the checkerboards. For the exact same checkerboard positions used for camera calibration, we have projected four fringe patterns onto the checkerboard, and have used the four-phase shift algorithm to relate the detected corner positions of the camera to a phase values, which is how the projector coordinates can be obtained (because we know which pixel of the projector projects which phase value).

Answer the following questions in 2-3 lines:

- What are the assumptions involved in the four-phase shift method? What are the benefits and drawbacks? (*Hint: Think in terms of external lighting conditions, movement of scene.*)
- Besides the four-phase shift method, other methods can also obtain camera-projector correspondence. Find one such method. In 2-3 lines, explain the method and its advantages or disadvantages over the four-phase shift (NOTE: You do not need to write code for this).
- Using the in-built function **undistortImage()** in MATLAB and **undistort()** in Python OpenCV, undistort all of the provided checkerboard images: “Data/Calibration/NoPattern”. Redetect the checker corners of the undistorted checker pattern. For the first checker pattern, plot a figure showing the detected corners from the first checkerboard image before undistortion (same as in Question 1b), and *on the same plot*, plot the detected corners from the first checkerboard after removing image distortion.
- Now we use the obtained undistorted checker corner locations to calibrate the projector. First, we need to undistort the images of the checkerboard that has a fringe pattern projected onto. These images are collected inside “Data/Calibration/Patterns”. You will calculate the phase map of the projected pattern onto each checker by implementing the four-phase shift algorithm. For each checker position, the first four fringe images (filename suffix is 1-4) are horizontal fringes, and the following four images (filename suffix is 5-8) are vertical fringes. Remember, the checker size is 15 mm. Similar to camera calibration, use the “**showExtrinsics()**” function to display the checkerboard planes, but now *with respect to the projector*.

Now use the four-phase shift algorithm to calculate the phase map for each checkerboard position and ultimately find camera-projector correspondence *at the detected checker corners*. As in the Deflectometry homework, you need to know that the period of the projected sinusoid in projector pixels along the vertical (rows) and horizontal (columns) directions is constant. The projector resolution is 2160 rows x 3840 columns. The projected fringe patterns in the horizontal direction (columns) range from $[0, 2\pi]$. Keeping the pixels per period constant along the vertical direction, the projected fringe pattern along the vertical direction (rows) ranges from $[0, 1.125\pi]$, since $2\pi * \frac{2160 \text{ pixels}}{3840 \text{ pixels}} = 1.125\pi$.

Plot the evaluated phase values at the detected checker corners for two checkerboard positions of your choice.

Question 3. Point cloud reconstruction (20 points). Consider what we have calculated so far. For each checkerboard pattern (world coordinates: $\hat{x}$), we have two projection matrices $\mathbf{P} = \mathbf{K} [\mathbf{R} | \mathbf{T}]$, for the camera and the projector projection metrics which we can denote as P_c and P_p , respectively. In other words, we have two equations relating the world coordinates to two different coordinates (camera and projector):

- i) $x_c = P_c \hat{x}$
- ii) $x_p = P_p \hat{x}$
- iii) Manipulate these equations to obtain the form

$$A\hat{x} = b,$$

where

$$\hat{x} = \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix},$$

and

$$b = \begin{bmatrix} x_c P_c^{34} - P_c^{14} \\ y_c P_c^{34} - P_c^{24} \\ x_p P_p^{34} - P_p^{14} \end{bmatrix},$$

where x_w, y_w, z_w are world coordinates, x_c, y_c, z_c are the camera coordinates, and x_p, y_p, z_p are projector coordinates.

Derive and report the expression for A. Refer to the lecture slides if you need help.

- a) For the “David” scene measurement, whose files are located inside “Data/Reconstruction/Patterns/David”, use your derived equation above to calculate the world coordinate $\hat{x}$ for each camera pixel. This is your point cloud! Obtain and display this point cloud. In MATLAB, convert the 3xN vector into a point cloud object, then display it using `pcshow()`. In Python, import open3d library and run:

`open3d.visualization.draw_geometries()`.

*Note: Since David’s background is quite noisy, you will only get a good result if you apply a mask by selecting just the David region. You can use this mask during reconstruction - telling your algorithm to obtain $\hat{x}$ for some x_p only if the corresponding camera pixel x_c, y_c belongs to the mask.

Load the mask “Data/DavidMask.png” which has the David bust pixels marked 1 and all others 0. Comment on the quality of reconstruction and anything else that you notice.

- b) **Fun and Bonus Points (does not need to be answered to get full points for this HW):** Export point cloud as ***.ply** file using `pcwrite()` in MATLAB and equivalently `open3d.io.write_point_cloud()` in Python. Download the software “Cloud Compare” (<https://www.danielgm.net/cc/release/>) and import the ***.ply** file. Generate normals for the point cloud using **Edit>Normals> Compute**. Now perform Poisson reconstruction using **Plugins>PoissonRecon**. What do you see? What looks different? Why? Include images of your results in the report. **(5 points)**

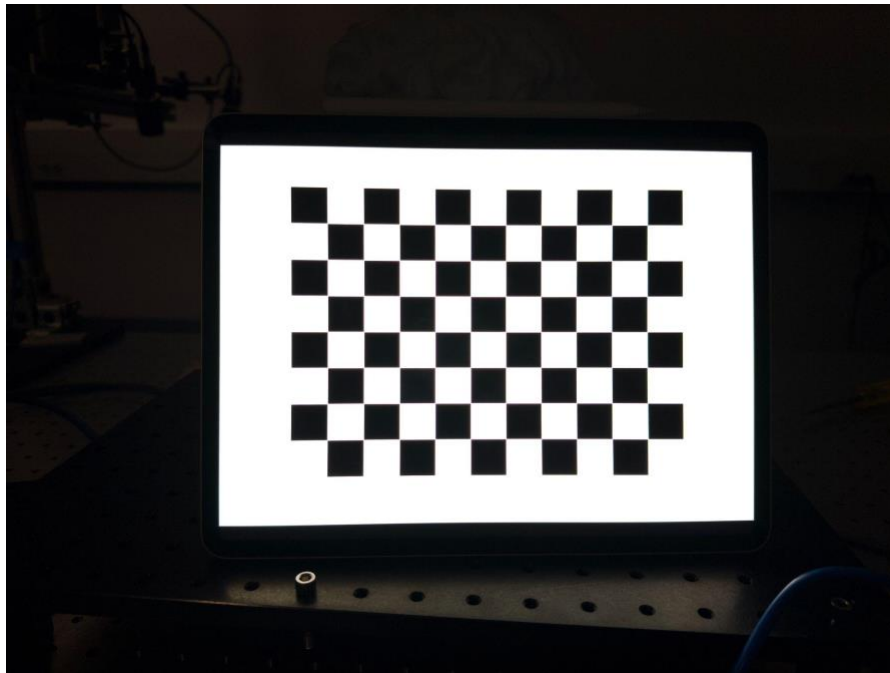
Part 2 (50 pts)

In this part, you will build a simple version of a passive stereo 3D sensor with your mobile phone camera. A standard passive stereo 3D sensor requires two identical cameras and precise calibration. Here we use one camera (your phone camera) and a simplified procedure. During some image capture processes, the camera needs to remain absolutely stationary. For this reason, you need to use the tripod again and Bluetooth remote from the Deflectometry homework. The detailed instructions for part 2 are written below. The way how this part is set up is that the instructions for each process are explained first. You should read this carefully and implement it accordingly. At the end of each section, there will be one or more questions that will test your gathered understanding and will ask you to plot some results to show that you have implemented everything successfully.

Instructions:

1. Camera intrinsic calibration:

- a) Prepare a checkerboard pattern. Use “*Pattern Generator*” on the website calib.io, and select “*Target Type*” as “*Checkerboard*”. Make sure that the number of rows and columns includes one odd number and one even number. This ensures that the software can correctly determine the orientation of the checkerboard. Set the checker size (in mm) of the checkerboard pattern to ensure that the entire pattern is visible within your phone camera's field of view (FOV). The checkerboard should fill most of your camera's FOV, but should not be larger (see example image below):



You can either print the checkerboard on a piece of paper or display the checkerboard on your computer screen. If you want to print the checkerboard: Export the pattern as a PDF and print it. Make sure the printed page does not get bent. For best results, the checkerboard should be as flat and straight as possible. To achieve this, you can mount the pattern on a straight foam board or a rigid poster board. Apply a minimal amount of adhesive around the outer edges of the printed sheet—this avoids wrinkles caused by excess moisture. Carefully attach the pattern to the board, avoiding any creases. Once the board is ready (and the glue has dried), **use a ruler or caliper to measure the actual physical size of one square, since printers may introduce slight scaling errors. This is very important to get good results!**

If you plan to display the checkerboard on a screen, find out your screen's PPI (pixels per inch), then determine how many pixels each checker spans (e.g. take a screenshot of your screen and display it in MATLAB). From that, you can calculate the checker size in millimeters, which will be used later as an input for camera calibration and 3D reconstruction. **Also measure the physical size of one checker on your screen with a ruler or caliper, you make sure you calculate everything correctly.**

b) Set up your phone camera in “pro-mode”, i.e., with manually fixed camera parameters. Similar to the Deflectometry homework, it is absolutely crucial that all camera parameters (focal length, exposure time, ISO, etc.) remain constant throughout the entire measurement sequence.

Later, we will move the checkerboard to different positions and orientations and take images accordingly—this will be explained in detail below. We need to carefully set the focal length to ensure that the checkerboard corners remain as sharp as possible in all images taken at different positions of the checkerboard. The method is as follows: you can define a “closest position” and a “farthest position” for your checkerboard and adjust your camera parameters so that the board appears sharp in both positions.

c) Then, capture a series of images of the checkerboard so they fill up the entire field of view. During this checkerboard capture process—i.e., during camera calibration—the camera must remain completely stationary. Use the tripod and a Bluetooth remote (exactly like you did in the Deflectometry homework).

Once again, it is critical that the camera parameters remain unchanged throughout the entire calibration process. We can change the position of the checkerboard, its distance from the camera, as well as its pitch and tilt angles. It is recommended to capture at least 15 images (the more, the better), making sure that the checkerboard corners are as sharp as possible. The space covered by the checkerboard positions defines the calibration volume, and in

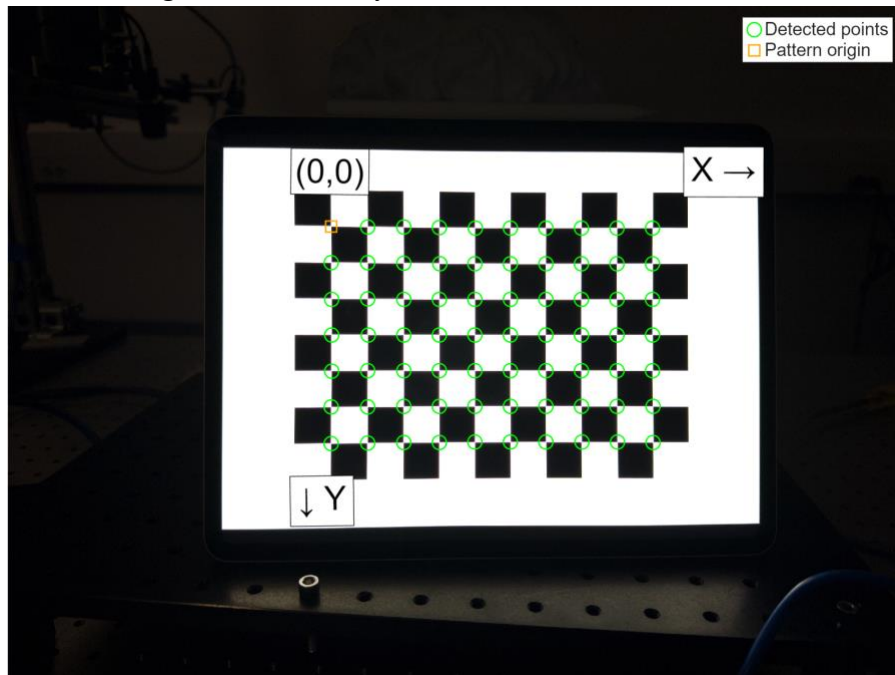
later steps, we will place the object to be measured within this volume. If you need some guidance on how to capture the images, look at the checkerboard image dataset that was provided for part 1.

Using the captured images, we can again estimate the camera's intrinsic parameters, just like we did in Part 1. Here, the size of the checkerboard square—which refers to the side length of each small square—is something we previously determined either manually or by calculating it from the PPI and the number of pixels.

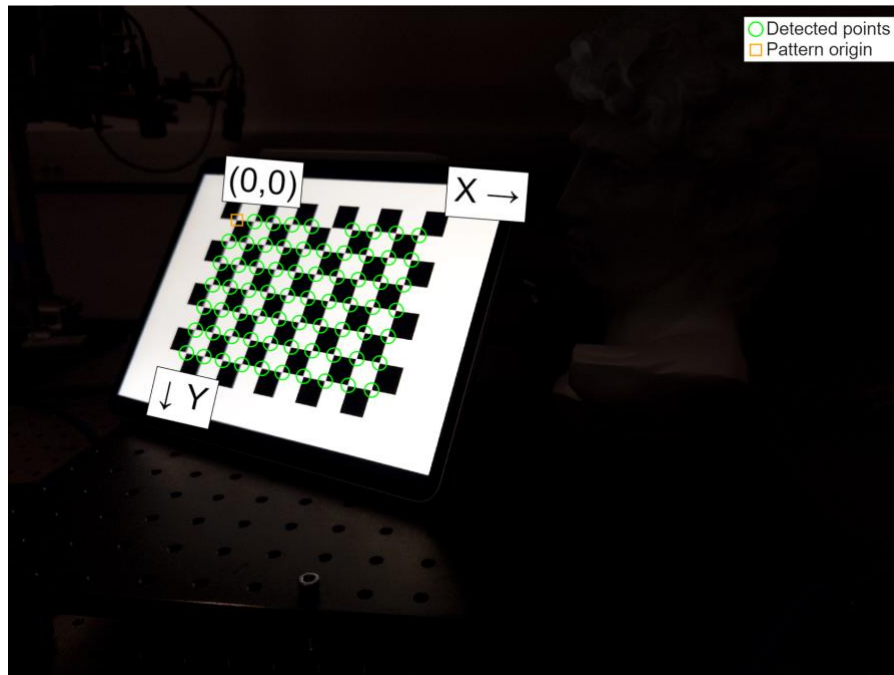
You can then inspect the detected checkerboard corners in each image. Again, if any image shows incorrect corner detection (for example, if all detected points are clustered together or if the detection is incomplete), it's best to remove those images from the calibration set.

Example:

Some Calibration Images with correctly detected checkerboard corners:



Calibration Images with incorrectly partially detected checker corners:



Question 1 (10 points). Perform the camera intrinsic calibration and visualize the result (the 3D plot of camera and used checkboards), also report the camera intrinsic parameters and reprojection error.

2. Image Acquisition:

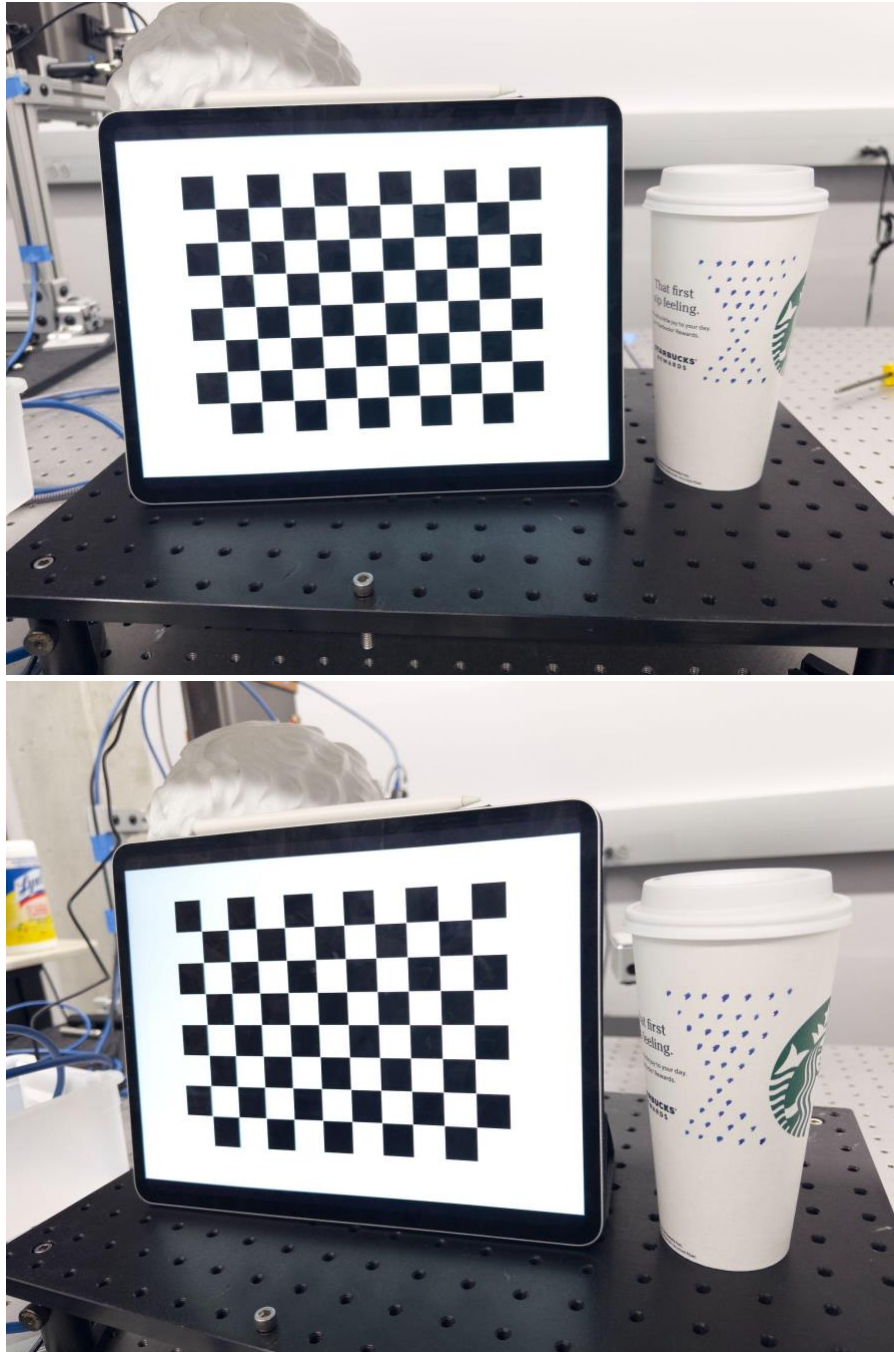
a) As discussed in class, passive stereo requires features in the captured images to establish correspondence between both cameras. For this reason (and to be on the safe side), you can apply some markers on the object you plan to reconstruct (for example, red dots).

In Part 1, we used the phase values from the projector and camera to establish correspondences. In this part, we will determine correspondence between the two images taken from different views based on these markers. In our example images below, we used a coffee cup as an object, and we painted a couple of blue points on the cup (see below). These matched points will be used in the next step to reconstruct 3D points in space. Of course, you should get creative and pick your own unique object. Instead of painting markers on the object, you can also use the structure of the surface itself (e.g., an imprinted pattern or features in a printed color image) as markers.

b) Since we are using a single camera to simulate a stereo pair by moving it between two positions, the checkerboard and object must remain fixed to allow accurate estimation of camera extrinsic parameters and subsequent 3D reconstruction. To achieve this, place a checkerboard and the target object within the same scene and within the calibration

volume, ensuring both are visible to the camera. The checkerboard will be used to estimate the camera's extrinsic parameters at both viewpoints. **Also, the camera parameters should remain the same as those used during the calibration step.** Capture the first image from the initial camera position (shown above). Then, without altering the position of the object or checkerboard, move the camera to a second viewpoint and capture the second image (shown below).

Example:



3. 3D Reconstruction:

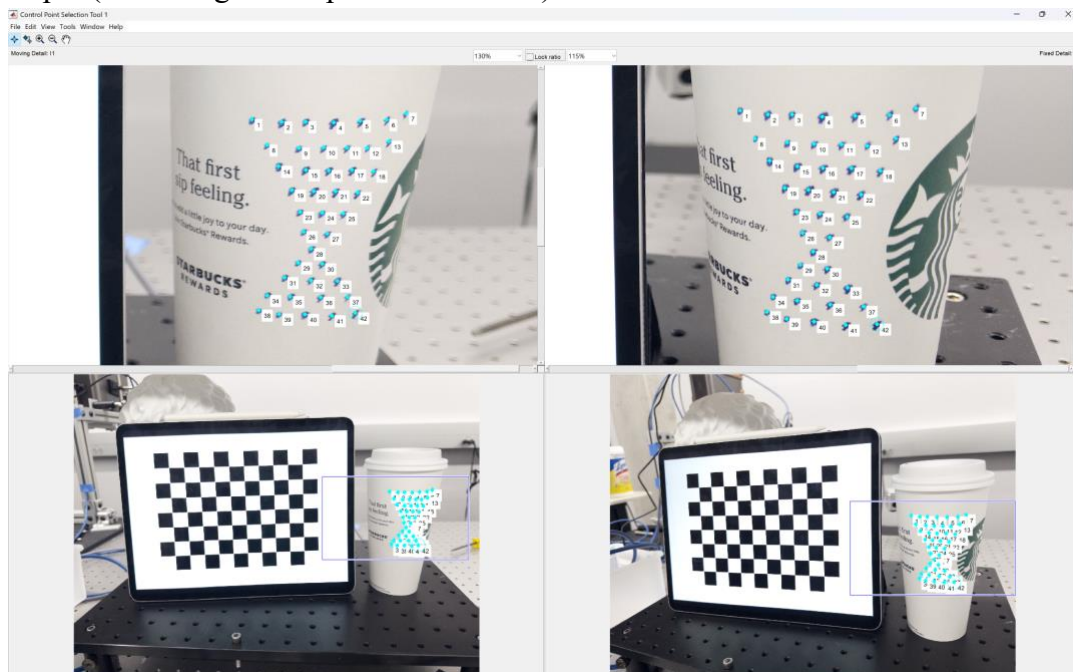
- First, undistort both images using the camera intrinsic parameters. You can use the function `undistortImage()` in MATLAB, or equivalently `undistort()` in OpenCV Python.
- Then, detect the checkerboard corners to help estimate the camera's extrinsic parameters in each view.

Question 2 (10 points). Show two images captured from two camera positions. Use the MATLAB function `plotCamera()` to visualize the camera position and orientations. For Python, use `scatter()` to plot the camera origin and function `quiver()` to visualize the camera orientation), and use `scatter3()` in MATLAB, or equivalently `scatter3()` in Python to plot the checkerboard points in the same 3D plot. Show the 3D plot in the report. The relative positions should roughly match how you physically arranged them during the experiment. In other words, the distance between the camera centers at the two views should approximately be equal to the distance the camera moved between the two views. The unit is the same as that of the checker size, millimeters.

- Next, use the markers placed on the object to identify corresponding feature points in both images. You will need to label the corresponding feature points *by hand*.

*Hint: The MATLAB functions `cpselect()` may be helpful. In Python, `ginput()` in Matplotlib or `setMouseCallback()` in OpenCV.

Example (look at light blue points and labels):





d) With the camera extrinsic parameters and the matched object points, you can update the relevant parameters in your reconstruction function from Part 1. Alternatively, you may use the `triangulate()` function from MATLAB's Computer Vision Toolbox or equivalently `triangulatePoints()` in Python OpenCV.

Question 3 (20 points). Detect stereo correspondences and perform 3D reconstruction. For example, if your object is a cup and you have drawn a line of markers (e.g., red dots) along its surface, you can manually identify a set of corresponding pixels in the two images. Using the known camera's extrinsic parameters and these matched points, perform triangulation to reconstruct their 3D positions. If your object is a cup, the resulting 3D points should approximate a **spherical or cylindrical curve**, reflecting the geometry of the cup's surface. Plot your point cloud using the function `scatter3()` in MATLAB or `scatter()` in Python. Include an image of the point cloud shown from 2 different perspectives in your report.

Question 4 (10 points). How is your 3D reconstruction? If not perfect, what might be the issue?

Question 5 (5 points). Fun and extra points (does not need to be answered to get full points for this HW): Try to fit a surface to the reconstructed point cloud using any suitable method to obtain a smooth representation. Show the result. What are the problems you have encountered? What might have gone wrong?